

Régulation Numérique

Correcteur PID Numérique

Du modèle à l'implémentation réelle

Etienne Chassaing basé sur le document d'Alain Le Gall
ECE Paris

1. Exemple introductif — PID Analogique vs PID numérique
2. Synthèse et discrétisation
3. Transposition en temps discret
4. Codage d'un PID numérique
5. Régler un PID en conditions réelles
6. Faire fonctionner un PID en conditions réelles
7. Validation théorique
8. Résumé

1. Exemple introductif — PID Analogique vs PID numérique
2. Synthèse et discrétisation
3. Transposition en temps discret
4. Codage d'un PID numérique
5. Régler un PID en conditions réelles
6. Faire fonctionner un PID en conditions réelles
7. Validation théorique
8. Résumé

Exemple introductif : Régulation de pression pour la production d'eau pure

Backpressure regulator

- Régulation **purement mécanique** (ressort + membrane)
- Pas de capteur, pas de calculateur
- Robuste mais **peu flexible** : un seul point de consigne

Un « correcteur analogique » intégré dans le mécanisme.

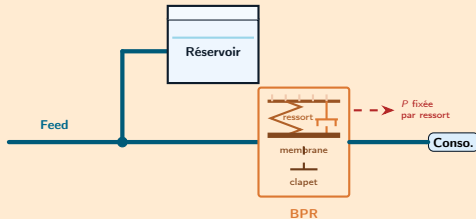
Valve de régulation + PID

- Capteur → CAN → **PID numérique** → CNA → actionneur
- Consigne modifiable, gains ajustables
- Implémentation : Anti-windup, filtrage, changement de mode...

Aujourd'hui : quasi 100% des PID industriels sont **numériques**.

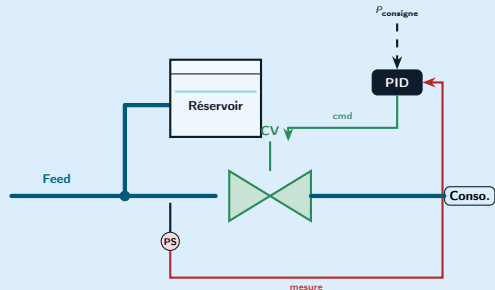
Exemple introductif Régulation de pression — Production d'eau pure

Solution 1 : Backpressure regulator



- Consigne fixée **mécaniquement**
- Pas de capteur, pas de signal électrique
- Aucun ajustement en fonctionnement

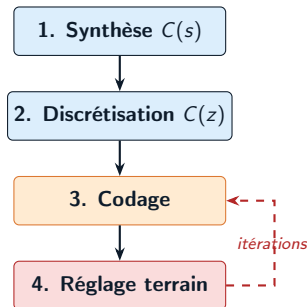
Solution 2 : Valve de régulation + PID



- Capteur (PS) mesure P en continu
- PID numérique pilote la vanne (CV)
- Consigne **ajustable**, alarmes, filtrage...

Pour être numérique, un PID a besoin :

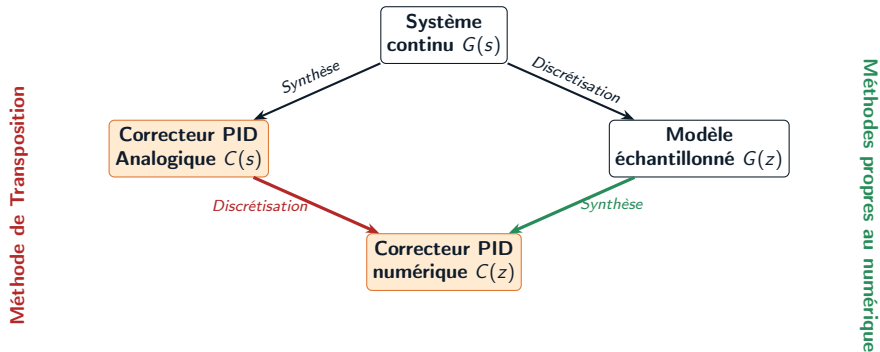
1. D'être **discrétisé** : passage $C(s) \rightarrow C(z)$ avec un choix de méthode et de T_e
2. D'être **implémenté** : algorithme récurrent, choix de la structure (parallèle, série, standard), anti-windup, dirty-derivative ...
3. D'être **réglé en conditions réelles** : le réglage théorique n'est qu'un point de départ



Le réglage terrain est **la partie la plus longue**. C'est là que se joue la différence entre un PID qui "marche" et un qui remplit toutes les spécifications.

1. Exemple introductif — PID Analogique vs PID numérique
2. Synthèse et discrétisation
3. Transposition en temps discret
4. Codage d'un PID numérique
5. Régler un PID en conditions réelles
6. Faire fonctionner un PID en conditions réelles
7. Validation théorique
8. Résumé

Les 2 voies pour la synthèse de correcteurs numériques



- La méthode de synthèse dépend en général du type de modèle disponible :
 - ▶ Avec un modèle continu, les spécifications sont plus facilement interprétables et **les méthodes de synthèse analogique sont bien maîtrisées** (placement pôles, ...)
 - ▶ Si on a un modèle discret, on peut également régler notre contrôleur directement en simulation, par exemple via Matlab

1. Synthèse d'un correcteur PID continu $C(s)$ par une méthode traditionnelle (Ziegler-Nichols, etc.) pour respecter le cahier des charges

$C(s)$

2. Transposition de $C(s)$ en correcteur numérique $C(z)$ pour s'approcher du comportement analogique



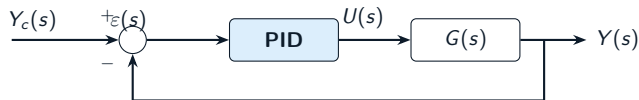
Avantages

- Méthodes analogiques éprouvées
- Spécifications facilement traduisibles
- Transition naturelle analogique $\rightarrow$ numérique

Attention

La stabilité avec le correcteur analogique ne garantit **pas** la stabilité avec le correcteur numérique (Système échantillonné, choix de T_e)

Rappel — Les formes du PID



PID parallèle

$$C(s) = k_p + \frac{k_i}{s} + k_d s$$

Gains indépendants k_p , k_i , k_d .

PID standard (idéal)

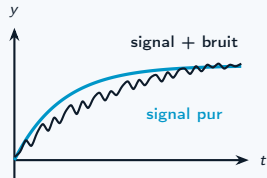
$$C(s) = K_p \left(1 + \frac{1}{T_i s} + T_d s \right)$$

Un seul gain K_p + constantes de temps.

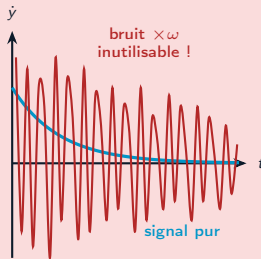
$$K_p = k_p, \quad T_i = \frac{k_p}{k_i}, \quad T_d = \frac{k_d}{k_p}$$

Rappel : Pourquoi filtrer le terme dérivé du PID ?

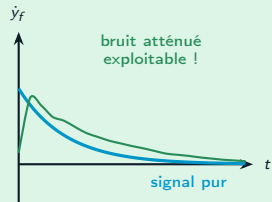
Signal mesuré $y(t)$



Dérivée brute $\frac{dy}{dt}$



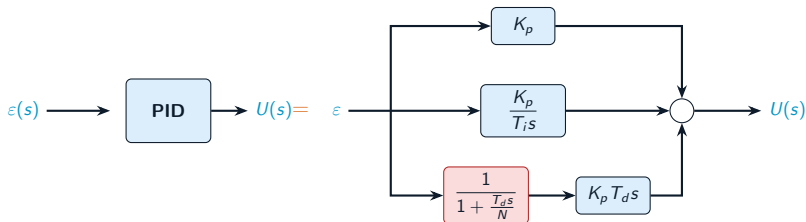
Dérivée filtrée $\frac{T_d s}{1 + T_d s/N}$



Dérivée pure $T_d s$ amplifie le bruit par un facteur $\propto \omega$ en haute fréquence. Plus la fréquence est élevée, plus l'amplification est forte.

Dérivée filtrée $\frac{T_d s}{1 + T_d s/N}$: se comporte comme un dérivateur en BF et comme un gain N en HF $\Rightarrow$ bruit borné.

Rappel — Schéma de régulation et PID réel



PID idéal

$$C(s) = K_p \left(1 + \frac{1}{T_i s} + T_d s \right)$$

PID réel (dérivée filtrée)

$$C(s) = K_p \left(1 + \frac{1}{T_i s} + \frac{T_d s}{1 + T_d s / N} \right)$$

avec $N \geq 5$ (typiquement $N = 10$)

PI : 90% des applications industrielles, PID : 95%. Réglage souvent itératif.

1. Exemple introductif — PID Analogique vs PID numérique
2. Synthèse et discrétisation
3. Transposition en temps discret
4. Codage d'un PID numérique
5. Régler un PID en conditions réelles
6. Faire fonctionner un PID en conditions réelles
7. Validation théorique
8. Résumé

PID réel continu de référence

$$C(s) = K_p \left(1 + \frac{1}{T_i s} + \frac{T_d s}{1 + T_d s / N} \right)$$

Euler avancée

$$s = \frac{z - 1}{T_e}$$

$$C(z) = K_p \left[1 + \frac{1}{T_i} \frac{T_e}{z - 1} + \frac{N(z - 1)}{z - (1 - \frac{NT_e}{T_d})} \right]$$

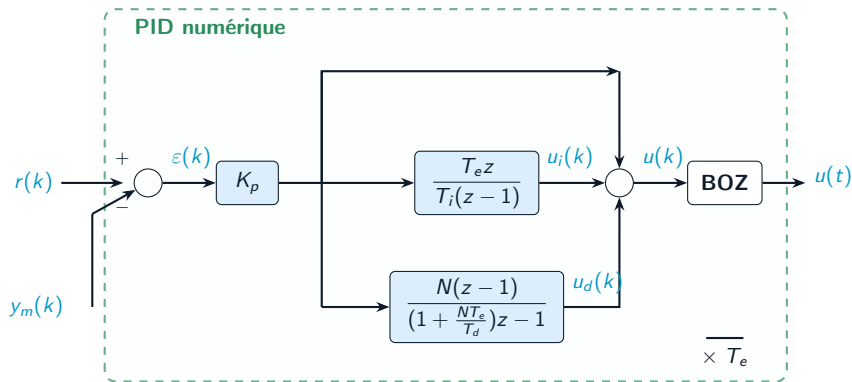
Euler retardée

$$s = \frac{1 - z^{-1}}{T_e} = \frac{z - 1}{T_e z}$$

$$C(z) = K_p \left[1 + \frac{1}{T_i} \frac{T_e z}{z - 1} + \frac{N(z - 1)}{(1 + \frac{NT_e}{T_d})z - 1} \right]$$

Détail d'un calcul :

Schéma d'implantation — Forme standard (approximation retardée)



Méthodes de transposition $C(s) \rightarrow C(z)$ — (2/2)

On peut voir les méthodes d'approximation comme les différentes façons de calculer une intégrale: rectangles gauches, rectangles droits, trapèzes.

Approximation de Tustin (bilinéaire)

$$s = \frac{2}{T_e} \frac{z-1}{z+1}$$

$$C(z) = K_p \left[1 + \frac{T_e}{2T_i} \frac{z+1}{z-1} + \frac{N(z-1)}{(1 + \frac{NT_e}{2T_d})z - (1 - \frac{NT_e}{2T_d})} \right]$$

En pratique :

- **Retardée** : formules les plus simples, la plus utilisée
- **Tustin** : meilleure fidélité fréquentielle

1. Exemple introductif — PID Analogique vs PID numérique
2. Synthèse et discrétisation
3. Transposition en temps discret
- 4. Codage d'un PID numérique**
5. Régler un PID en conditions réelles
6. Faire fonctionner un PID en conditions réelles
7. Validation théorique
8. Résumé

Pseudo code d'un PID numérique

Pseudo-code PID parallèle

```
# def Te, Ki, Kp, Kd, r
i = 0
e0 = e = 0
while running:
    y = ...                # read
                           measure
    e = r - y              #
                           compute error
    i += Ki*Te*e           # update
                           integral
    u = Kp*e + i + Kd/Te*(e-e0)
    e0 = e                 # save
                           error
```

Points clés

- K_p , K_i , K_d réglés indépendamment
- Intégrale mise à jour de manière **incrémentale**
- Dérivée par différence finie $\frac{e(k)-e(k-1)}{T_e}$

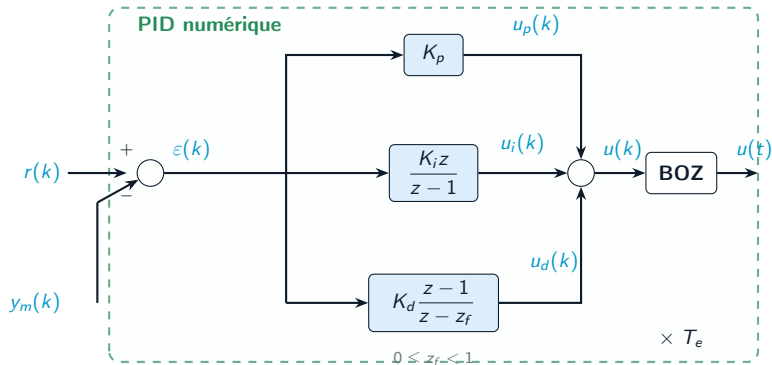
Ce code est assez **basique**. En conditions réelles, il faut y ajouter toutes les « astuces » de la section suivante.

Schéma bloc — Forme parallèle du PID numérique

Sous cette forme, le PID est un peu plus simple à régler empiriquement:

$$C(s) = K_p + \frac{K_i}{s} + K_d s$$

z_f se trouve en mettant le terme dérivée sous la bonne forme:



1. Exemple introductif — PID Analogique vs PID numérique
2. Synthèse et discrétisation
3. Transposition en temps discret
4. Codage d'un PID numérique
5. Régler un PID en conditions réelles
6. Faire fonctionner un PID en conditions réelles
7. Validation théorique
8. Résumé

Méthodes de réglage — Trois approches complémentaires

1. Maths / Modèles

- Placement de pôles
- Marges de gain/phase sur diagramme de Bode/Nyquist
- Méthodes analytiques

Nécessite un modèle fiable.

2. Intuition physique

- Raisonner sur les fréquences de résonance ($\sim$ Bode)
- Constantes de temps dominantes (système/actionneurs)
- Quel gain $\rightarrow$ quel effet

L'expérience se construit ici.

3. Trial & error

- Observer, modifier, re-tester
- **P** seul, puis D, puis I
- Documenter chaque essai
- **Méthode reine** sur site

Indispensable et inévitable.

Rappel : Ziegler-Nichols est par exemple un mélange des trois — une recette empirique fondée sur l'observation de la réponse du système, qui fournit un pré-réglage à affiner.

Rappel — Influence des actions P, I, D

Influence des actions P, I, D

	T_m	T_s	D	ε_s
$K_p \nearrow$	$\searrow$	$\nearrow$	$\nearrow$	$\searrow$
$K_i \nearrow$	$\searrow$	$\nearrow$	$\nearrow$	$= 0$
$K_d \nearrow$	$-$	$\searrow$	$\searrow$	$-$

T_{monte} : temps de montée, $T_{stabilisation}$: stabilisation,

D : dépassement, ε_s : erreur statique

Paramètres à régler

K_p , K_i ou T_i , K_d ou T_d , T_t , N

N souvent fixé à $N = 10$, $T_t \in [0,1 T_i ; T_i]$

Démarche : commencer P seul ($K_i = 0$, $K_d = 0$), augmenter K_p jusqu'à une réponse acceptable, puis ajouter D et I très progressivement.

Le réglage théorique n'est qu'un **pré-réglage**

Pourquoi ça marche rarement du premier coup

- Le modèle $G(s)$ est **approximatif** (linéarisé, simplifié, identifié sur un point de fonctionnement)
- Non-linéarités non modélisées (jeu, frottements, hystérésis, saturation)
- Perturbations réelles non prises en compte
- Bruit de mesure, retards capteurs/actionneurs
- Le système **change** (température, usure, charge)

L'expérience fait loi

Les méthodes de réglage donnent un **point de départ**. Ensuite :

- Observer la réponse réelle
- Ajuster les gains itérativement
- Tester sur différents points de fonctionnement
- Vérifier la robustesse aux perturbations

Les modèles fonctionnent en général mieux quand les systèmes sont naturellement lents.

Règle d'or : La théorie fournit un cadre de conception et de validation mais l'expérience fait foi.

1. Exemple introductif — PID Analogique vs PID numérique
2. Synthèse et discrétisation
3. Transposition en temps discret
4. Codage d'un PID numérique
5. Régler un PID en conditions réelles
- 6. Faire fonctionner un PID en conditions réelles**
7. Validation théorique
8. Résumé

Les astuces qui font fonctionner un PID en vrai

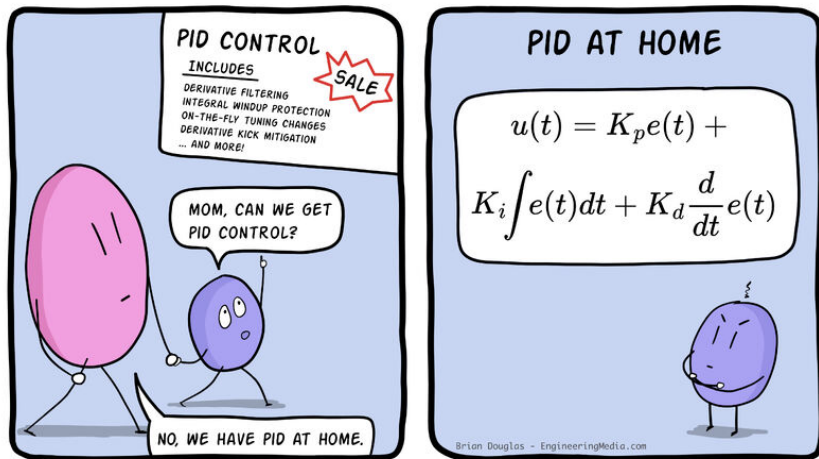


Figure: Les "astuces" du PID, Brian Douglas

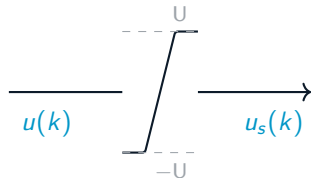
Les astuces qui font fonctionner un PID en vrai

Sans ces mécanismes, le PID échoue sur le terrain

Problème	Solution
Saturation actionneur	Anti-windup — boucle de retour sur l'intégrateur
Pics de commande	Dérivation de la sortie y_m et pas de la consigne r
Bruit amplifié par D	Filtrage de la dérivée : $\frac{T_d s}{1 + T_d s/N}$, $N \approx 5-20$
Bruit capteur	Filtrage de la mesure (passe-bas sur y_m)
Mauvais T_e	$T_e < T_c/10$, sinon dégradation et instabilité

Anti-windup — *Integral Reset*

- La partie intégrale peut entraîner des effets indésirables lorsque l'intégrateur se **gonfle** et que l'actionneur **sature**
- L'actionneur reste en butée, même lorsque la sortie du système varie



Saturation de la commande

On sature toujours la commande :

$$u_s(k) = \begin{cases} U & \text{si } u(k) > U \\ u(k) & \text{si } |u(k)| \leq U \\ -U & \text{si } u(k) < -U \end{cases}$$

On cesse à ce moment d'incrémenter la valeur de l'intégrale.

Remise à zéro de l'intégrale

Lors d'un **changement de consigne brutal**, l'intégrale accumulée devient inadaptée au nouveau point de fonctionnement. On la remet à zéro à l'instant k_0 du changement : $I(k_0) = 0$ Cela évite un dépassement excessif en transitoire.

Dérivation de la sortie, pas de la consigne

Le problème

Quand la consigne $r(k)$ subit un changement brusque (échelon), la dérivée de l'erreur produit un **pic de commande** (*derivative kick*) qui peut :

- Endommager les actionneurs
- Déstabiliser le système
- Saturer la commande

La solution

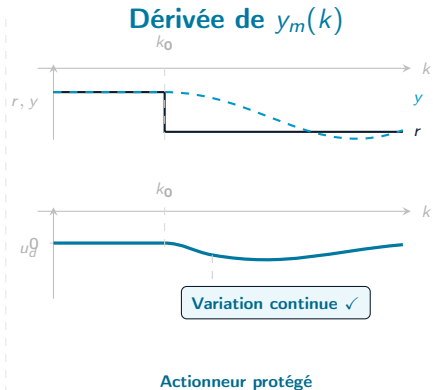
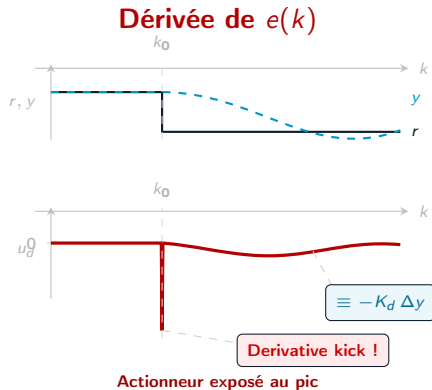
Dériver **uniquement la mesure** $y_m(k)$:

$$u_d(k) = -K_d \frac{y_m(k) - y_m(k-1)}{T_e}$$

La mesure varie de manière **continue** $\Rightarrow$ pas de pic.

En pratique : c'est la forme **systématiquement** utilisée dans les automates industriels.

Dérivation de la sortie, pas de la consigne — schéma



Forme sans dérivation de l'entrée (approximation retardée)

$$\varepsilon(k) = r(k) - y_m(k)$$

$$u(k) = K_p \varepsilon(k) + u_i(k) + u_d(k)$$

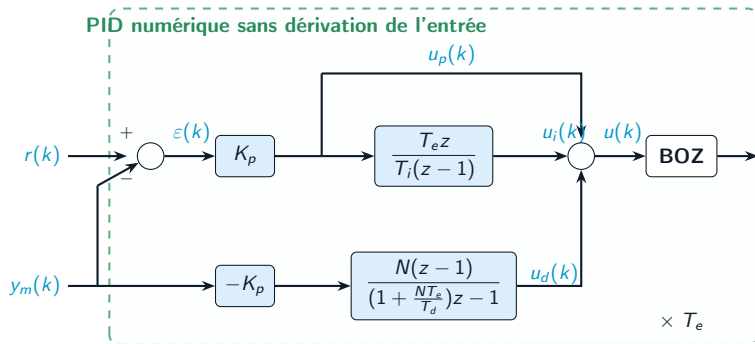
$$u_i(k) = u_i(k-1) + K_p \frac{T_e}{T_i} \varepsilon(k-1)$$

$$u_d(k) = \underbrace{\frac{1}{1 + \frac{NT_e}{T_d}} u_d(k-1)}_{\text{filtrage}} - \underbrace{\frac{K_p N}{1 + \frac{NT_e}{T_d}} [y_m(k) - y_m(k-1)]}_{\text{calcul de la dérivée}}$$

La dérivée agit sur la **mesure** y_m (et non sur l'erreur ε), ce qui évite les pics de commande lors de changements de consigne brusques.

Forme sans dérivation de l'entrée (approximation retardée)

En pratique :



Filtrage de la mesure

On filtre toujours y_m **avant** le PID, exemple filtre du 1er ordre :

$$y_f(k) = \alpha y_f(k-1) + (1 - \alpha) y_m(k)$$

avec $\alpha = e^{-T_e/\tau_f}$.

Règle : $\tau_f \ll T_i$ pour ne pas dégrader les performances du PID. **Remarque :** Si $\alpha = 0.5$ on a un filtre à moyenne glissante.

Choix du filtre

On choisit un filtre adapté à nos capteurs et à la rapidité de notre système.

Attention:

Un filtrage trop agressif ajoute du **retard de phase**, ce qui réduit la marge de stabilité. C'est un compromis bruit / performance.

Plus intuitivement, on perd de l'information en filtrant (on devient aveugles à certaines fréquences)

Règle fondamentale

$$T_e < \frac{T_c}{10}$$

T_c = constante de temps dominante du correcteur.

Si T_e est trop grand :

- Perte de performance (retard numérique)
- Risque d'instabilité

Le BOZ ajoute un retard $\approx T_e/2$

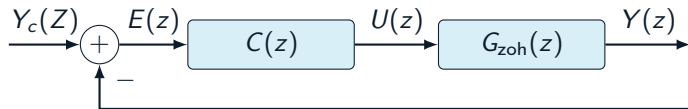
Bonnes pratiques

- $T_e \approx T_c/10$ à $T_c/20$ est un bon compromis
- La fréquence de **mesure** est souvent plus rapide que celle de **contrôle** (suréchantillonnage pour filtrage)
- Sur microcontrôleur : vérifier que le calcul tient dans un pas T_e

1. Exemple introductif — PID Analogique vs PID numérique
2. Synthèse et discrétisation
3. Transposition en temps discret
4. Codage d'un PID numérique
5. Régler un PID en conditions réelles
6. Faire fonctionner un PID en conditions réelles
- 7. Validation théorique**
8. Résumé

Validation théorique du correcteur $C(z)$

Pour valider théoriquement le correcteur PID discret obtenu, on doit étudier la stabilité du **système bouclé discret**, en remplaçant le système par son modèle échantillonné $G_{zoh}(z)$.



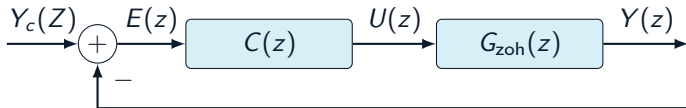
Fonction de transfert en boucle fermée

$$\frac{Y(z)}{Y_c(Z)} = \frac{C(z) G_{zoh}(z)}{1 + C(z) G_{zoh}(z)}$$

La stabilité s'analyse par la position des **pôles de la BF** dans le plan z .

Validation théorique du correcteur $C(z)$

Pour valider théoriquement le correcteur PID discret obtenu, on doit étudier la stabilité du **système bouclé discret**, en remplaçant le système par son modèle échantillonné $G_{zoh}(z)$.



Nouvelles dynamiques introduites par l'échantillonnage

La stabilité en continu de $C(s) \rightarrow G(s)$ **ne garantit pas** la stabilité de $C(z) \rightarrow G_{zoh}(z)$:

- $G_{zoh}(z)$ fait apparaître des **zéros de discrétisation** absents de $G(s)$
- Le ZOH introduit un retard pur $\approx T_e/2$, qui **réduit les marges de stabilité**
- Plus T_e augmente, plus ces effets sont marqués $\rightarrow$ risque de déstabilisation

1. Exemple introductif — PID Analogique vs PID numérique
2. Synthèse et discrétisation
3. Transposition en temps discret
4. Codage d'un PID numérique
5. Régler un PID en conditions réelles
6. Faire fonctionner un PID en conditions réelles
7. Validation théorique
8. Résumé

La chaîne complète, en V

1. **Modéliser** $\rightarrow G(s)$
2. **Synthétiser** $\rightarrow C(s)$
3. **Discrétiser** $\rightarrow C(z)$ (Tustin ou retardée)
4. **Coder** l'algorithme récurrent
5. **Protections** : anti-windup, dérivée sur y_m , filtrage, saturation
6. **Réglage terrain** : trial & error
7. **Validation** : critère de stabilité, simulations, expérience en cas limites

Discrétisation — À retenir

- **Retardée** : formules simples, la plus utilisée
- **Tustin** : meilleure fidélité fréquentielle
- $T_e < T_c/10$ conseillé
- Stabilité analogique $\nrightarrow$ stabilité numérique
- BOZ/ZOH : retard $\approx T_e/2$

Ce qui fera de vous un bon ingénieur : savoir **coder**, **régler**, et **debugger** sur un vrai système — en faisant des allers-retours entre théorie et pratique.